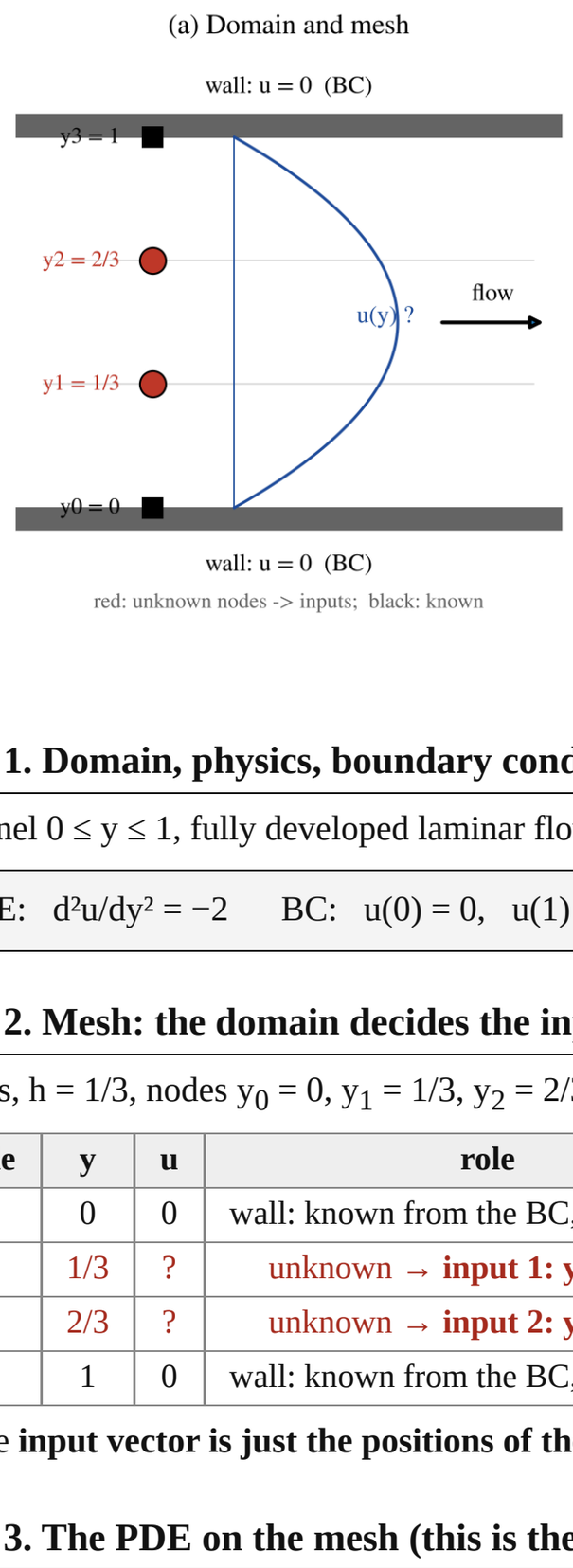
The network in one dimension, by hand

Flow between two plates: 3 cells, 2 unknowns, 2 hidden nodes. Every number below can be checked with a calculator (tanh 1 = 0.76159).



Step 1. Domain, physics, boundary conditions

Channel $0 \le y \le 1$, fully developed laminar flow (pressure drives, viscosity resists):

PDE: $d^2u/dy^2 = -2$ BC: $u(0) = 0$, $u(1) = 0$ (no slip) exact answer: $u = y(1 - y)$

Step 2. Mesh: the domain decides the inputs

3 cells, $h = 1/3$, nodes $y_0 = 0$, $y_1 = 1/3$, $y_2 = 2/3$, $y_3 = 1$.

node	y	u	role
0	0	0	wall: known from the BC, not an input
1	1/3	?	unknown → **input 1: y = 0.3333**
2	2/3	?	unknown → **input 2: y = 0.6667**
3	1	0	wall: known from the BC, not an input

So the **input vector is just the positions of the unknown nodes**: x = (1/3, 2/3).

Step 3. The PDE on the mesh (this is the "loss"; the BCs go in here)

Central difference at each unknown node: $(u_{j-1} - 2u_j + u_{j+1})/h^2 + 2 = 0$. Put in the wall values $u_0 = u_3 = 0$ (the BCs) and $1/h^2 = 9$:

$r_1 = 9(-2u_1 + u_2) + 2$ $r_2 = 9(u_1 - 2u_2) + 2$ (we want $r_1 = r_2 = 0$)

Classic CFD by hand: $-2u_1 + u_2 = -2/9$ and $u_1 - 2u_2 = -2/9$ ⇒ by symmetry $u_1 = u_2$, so $-u_1 = -2/9$ ⇒ **$u_1 = u_2 = 2/9 = 0.2222$**.

Step 4. Forward pass: input → hidden layer → output

Hidden nodes (weights fixed, never trained): $a_1 = \tanh(3y - 1)$, $a_2 = \tanh(3y - 2)$. Output: $u = z_1a_1 + z_2a_2$.

input y	$s_1 = 3y - 1$	$a_1 = \tanh s_1$	$s_2 = 3y - 2$	$a_2 = \tanh s_2$	output u
1/3	0	0	−1	−0.76159	$u_1 = -0.76159\, z_2$
2/3	1	0.76159	0	0	$u_2 = 0.76159\, z_1$

Start with $z_1 = z_2 = 0$: $u_1 = u_2 = 0$ (no flow). Residual: $r_1 = r_2 = 2 \neq 0$, so the physics says this is wrong.

Step 5. Find the output weights z from the physics (one Newton step)

Put the network output into the residuals ($a = 0.76159$):

$r_1 = 9(2a\, z_2 + a\, z_1) + 2 = 6.854\, z_1 + 13.709\, z_2 + 2$ $r_2 = 9(-a\, z_2 - 2a\, z_1) + 2 = -13.709\, z_1 - 6.854\, z_2 + 2$

Set both to zero. With $k = 2/(9a) = 2/6.854 = 0.29179$: $z_1 + 2z_2 = -k$, $2z_1 + z_2 = k$ ⇒ **$z_1 = 0.29179$, $z_2 = -0.29179$**.

(This is exactly one Newton step: Jacobian $J = \partial r/\partial z = [[6.854, 13.709], [-13.709, -6.854]]$, $\Delta z = -J^{-1}r$. The equations are linear, so one step is exact.)

Step 6. Output and check

node	network: $u = z_1a_1 + z_2a_2$	classic CFD (hand)	exact y(1−y)	residual r
y = 1/3	0.29179·0 + (−0.29179)(−0.76159) = **0.2222**	0.2222	0.2222	0
y = 2/3	0.29179·0.76159 + (−0.29179)·0 = **0.2222**	0.2222	0.2222	0

What the network does: the domain and mesh give the inputs (positions of the unknown nodes). The fixed hidden layer turns each position into numbers a_1, a_2. The output is a weighted sum. The weights z are not learned from data; they are chosen so that the discretised PDE, with the BCs inside it, is satisfied. That is why the network gives exactly the classic CFD answer.

Notes. (1) Here the answer also equals the exact one because central differences are exact for a parabola; in general both the network and classic CFD carry the same mesh error, which falls as the mesh is refined. (2) Other hidden weights, e.g. $a_1 = \tanh(6y - 3)$, $a_2 = \tanh(-3y + 1)$, give different z but the same u = 0.2222. (3) The real code does the same with (x, y, z) inputs, thousands of nodes and the Navier–Stokes equations; the script spiral_1d_hand.py reproduces every number on this page.